

Moner Bondhu — Full AI Mental Health Chatbot

Features

- AI Chatbot
 - Beautiful UI
 - Responsive Design
 - Dark Mode
 - Typing Animation
 - Gemini AI Support
 - Emergency Support Message
 - Bengali + English Support
 - Chat History
 - Mobile Friendly
-

Step 1 — Create Project Folder

```
mkdir moner-bondhu  
cd moner-bondhu
```

Step 2 — Initialize Project

```
npm init -y  
npm install express cors dotenv axios
```

Step 3 — Project Structure

```
moner-bondhu/  
|  
├─ server.js  
├─ .env  
├─ package.json  
├─ public/  
|   └─ index.html
```

```
| | style.css
| | script.js
```

Step 4 — Create .env File

```
GEMINI_API_KEY=YOUR_GEMINI_API_KEY
PORT=3000
```

Step 5 — Get Gemini API Key

Go to:

<https://aistudio.google.com/app/apikey>

Create API Key.

Paste it inside `.env`

Step 6 — server.js

```
require("dotenv").config();

const express = require("express");
const cors = require("cors");
const axios = require("axios");
const path = require("path");

const app = express();

app.use(cors());
app.use(express.json());
app.use(express.static(path.join(__dirname, "public")));

const GEMINI_API_KEY = process.env.GEMINI_API_KEY;

app.post("/chat", async (req, res) => {
  try {
    const userMessage = req.body.message;
```

```

    const prompt = `
You are Moner Bondhu, a supportive AI mental health assistant.

Rules:
- Be kind and supportive.
- Reply in Bengali if user uses Bengali.
- Never give dangerous advice.
- Encourage positivity.
- Keep answers human friendly.
- If user seems suicidal, encourage contacting family or professionals.

User Message:
${userMessage}
`;

    const response = await axios.post(
      `https://generativelanguage.googleapis.com/v1beta/models/gemini-1.5-
flash:generateContent?key=${GEMINI_API_KEY}`,
      {
        contents: [
          {
            parts: [
              {
                text: prompt,
              },
            ],
          },
        ],
      },
    );

    const reply =
      response.data.candidates[0].content.parts[0].text;

    res.json({ reply });
  } catch (error) {
    console.log(error.message);

    res.json({
      reply:
        "আমি এখন একটু সমস্যায় আছি 🙄 কিছুক্ষণ পরে আবার চেষ্টা করো।",
    });
  }
});

app.listen(process.env.PORT, () => {

```

```
console.log(`Server running on port ${process.env.PORT}`);
});
```

Step 7 — public/index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Moner Bondhu</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <div class="container">

    <div class="top-bar">
      <h1>💙 Moner Bondhu</h1>
      <button id="darkBtn">🌙</button>
    </div>

    <div id="chatBox" class="chat-box"></div>

    <div class="input-area">
      <input
        type="text"
        id="message"
        placeholder="Type your feelings..."
      />

      <button onclick="sendMessage()">Send</button>
    </div>

    <div class="emergency">
      🚨 If you are in danger or having self-harm thoughts,
      please contact trusted family members or professional help.
    </div>

  </div>

  <script src="script.js"></script>
```

```
</body>
</html>
```

Step 8 — public/style.css

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

body {
  background: #f0f4ff;
  height: 100vh;
  display: flex;
  justify-content: center;
  align-items: center;
  transition: 0.3s;
}

.container {
  width: 95%;
  max-width: 500px;
  background: white;
  border-radius: 20px;
  overflow: hidden;
  box-shadow: 0 10px 30px rgba(0,0,0,0.1);
}

.top-bar {
  background: #4f46e5;
  color: white;
  padding: 15px;
  display: flex;
  justify-content: space-between;
  align-items: center;
}

.top-bar button {
  background: white;
  border: none;
  padding: 8px 12px;
  border-radius: 10px;
```

```
    cursor: pointer;
}

.chat-box {
  height: 500px;
  overflow-y: auto;
  padding: 15px;
  display: flex;
  flex-direction: column;
  gap: 10px;
}

.message {
  padding: 12px;
  border-radius: 15px;
  max-width: 80%;
  line-height: 1.5;
}

.user {
  background: #4f46e5;
  color: white;
  align-self: flex-end;
}

.bot {
  background: #e5e7eb;
  color: black;
  align-self: flex-start;
}

.input-area {
  display: flex;
  border-top: 1px solid #ddd;
}

.input-area input {
  flex: 1;
  padding: 15px;
  border: none;
  outline: none;
}

.input-area button {
  padding: 15px;
  border: none;
  background: #4f46e5;
  color: white;
}
```

```
    cursor: pointer;
}

.emergency {
  padding: 10px;
  background: #fee2e2;
  color: #991b1b;
  font-size: 13px;
  text-align: center;
}

.dark {
  background: #111827;
}

.dark .container {
  background: #1f2937;
}

.dark .bot {
  background: #374151;
  color: white;
}

.dark input {
  background: #111827;
  color: white;
}
```

Step 9 — public/script.js

```
const chatBox = document.getElementById("chatBox");
const input = document.getElementById("message");
const darkBtn = document.getElementById("darkBtn");

function addMessage(text, className) {
  const div = document.createElement("div");
  div.classList.add("message", className);
  div.innerText = text;
  chatBox.appendChild(div);

  chatBox.scrollTop = chatBox.scrollHeight;
}
```

```

async function sendMessage() {
  const message = input.value.trim();

  if (!message) return;

  addMessage(message, "user");

  input.value = "";

  const typing = document.createElement("div");
  typing.classList.add("message", "bot");
  typing.innerText = "Typing...";

  chatBox.appendChild(typing);

  try {
    const response = await fetch("/chat", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ message }),
    });

    const data = await response.json();

    typing.remove();

    addMessage(data.reply, "bot");

    saveChat();
  } catch (error) {
    typing.remove();

    addMessage("Something went wrong 😞", "bot");
  }
}

input.addEventListener("keypress", (e) => {
  if (e.key === "Enter") {
    sendMessage();
  }
});

function saveChat() {
  localStorage.setItem("chatData", chatBox.innerHTML);
}

```



```
function loadChat() {  
  const data = localStorage.getItem("chatData");  
  
  if (data) {  
    chatBox.innerHTML = data;  
  }  
}  
  
loadChat();  
  
// Dark Mode  
  
darkBtn.addEventListener("click", () => {  
  document.body.classList.toggle("dark");  
});
```

Step 10 — Run Project

```
node server.js
```

Step 11 — Open Browser

```
http://localhost:3000
```

Step 12 — Upload To GitHub

```
git init  
git add .  
git commit -m "Moner Bondhu AI Chatbot"
```

Create repository on:

<https://github.com>

Then:

```
git remote add origin YOUR_GITHUB_REPO_LINK
git branch -M main
git push -u origin main
```

Step 13 — Deploy Online Free

You can deploy on:

- Render
- Railway
- Vercel

Recommended:

<https://render.com>

Future Features

You can later add:

- Voice Chat
 - Login System
 - User Accounts
 - Firebase Database
 - Emotion Detection
 - AI Memory
 - Android App
 - Video Call Support
 - Doctor Appointment
 - SOS Button
-

Important

Never upload your `.env` file publicly.

Create `.gitignore`

```
node_modules
.env
```

Finished

Your AI Mental Health Chatbot is ready ❤️